DeepSeek V4.1 Flash Engram

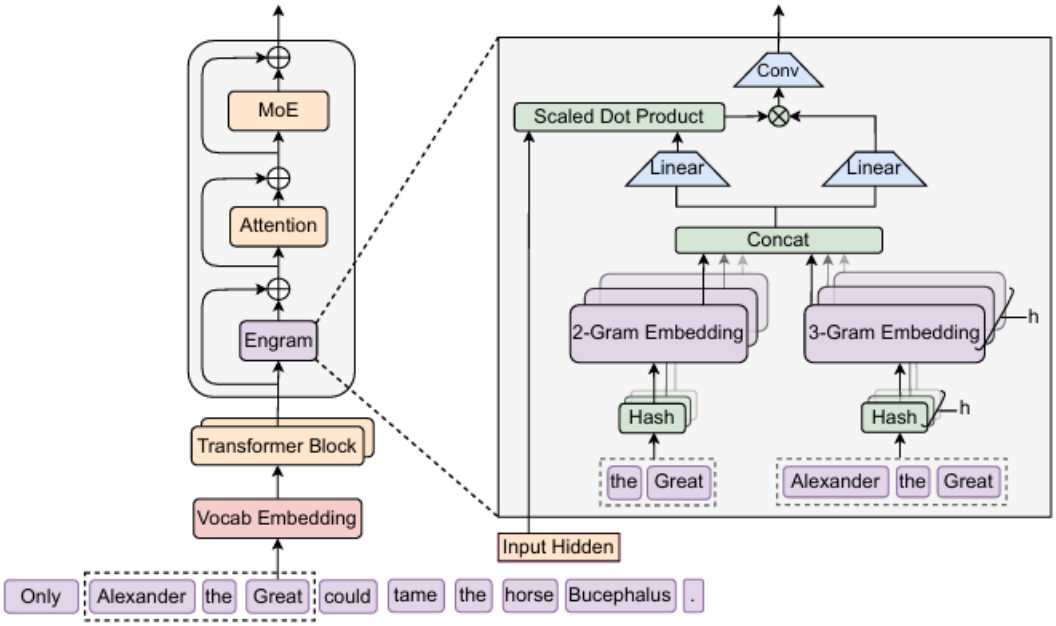


Figure 1: Engram 架构细节图。左侧展示Engram 在Transformer Block 中的位置：输入经Vocab Embedding 和前序Transformer Block 后进入当前块，当前块内部顺序为Engram → Attention → MoE。右侧展开Engram 内部核心组件：对当前位置构造n-gram 后缀（实际配置$n = 2, 3, 4$，图中示意2-Gram/3-Gram），经Hash 映射查表、Concat 拼接、Linear 投影得到K/V；以当前隐藏状态为Q，与K 做Scaled Dot Product 门控，再作用于V，最后经Conv1D + SiLU 与残差注入回隐藏状态。

1 真实配置

项目	真实V4.1-Flash
压缩词表	99,092
后缀n-gram	$n = 2, 3, 4$
哈希头数	8
head_dim	256
每行存储	256 字节FP8
Engram 层	2 层
表行数	每层约3.84 亿行

真实中，每个token 位置在每个Engram 层取：3 个n-gram 阶数×8 个哈希头= 24 行，每行256 字节FP8，拼成6144 字节记忆向量。

2 输入与压缩

原始token：`["The", "capital", "of", "France", "is"]`。压缩映射：The→the(1)，capital→capital(2)，of→of(3)，压缩ID 序列为`[1, 2, 3, 4, 5]`，当前位置是最后一个token `is`，Engram 在此介入。

3 构造后缀n-gram

在is 位置取最近2、3、4 个压缩token：2-gram (4,5)，3-gram (3,4,5)，4-gram (2,3,4,5)。

4 多头部哈希与检索

真实中，每个$(n, head)$ 对分配一个素数p 和偏移off，保证桶区间互不重叠。哈希公式：h = (id_1 * m_1 XOR id_2 * m_2 XOR ...) mod p，bucket = off + h。以下用微缩数值展示计算逻辑：

```
(2,0) n-gram (4,5):  4*31=124, 5*37=185, XOR=197, mod101=96, off0 -> bucket 96
(2,1) n-gram (4,5):  4*41=164, 5*43=215, XOR=115, mod103=12, off101 -> bucket 113
(3,0) n-gram (3,4,5): 3*17=51, 4*19=76, 5*23=115, XOR=12, mod107=12, off204 ->
    bucket 216
(3,1) n-gram (3,4,5): 3*29=87, 4*31=124, 5*37=185, XOR=146, mod109=37, off311 ->
    bucket 348
(4,0) n-gram (2,3,4,5): 2*13=26, 3*17=51, 4*19=76, 5*23=115, XOR=22, mod113=22,
    off420 -> bucket 442
(4,1) n-gram (2,3,4,5): 2*11=22, 3*13=39, 4*17=68, 5*19=95, XOR=42, mod127=42,
    off533 -> bucket 575
```

命中的6 个桶为96, 113, 216, 348, 442, 575。查表得每行4 维向量：

桶索引	行向量
96	[0.90, 0.10, 0.80, 0.20]
113	[0.70, 0.00, 0.60, 0.10]
216	[0.80, 0.20, 0.90, 0.30]
348	[0.60, 0.10, 0.70, 0.20]
442	[0.95, 0.00, 0.85, 0.10]
575	[0.50, 0.30, 0.40, 0.20]

拼接得到24 维记忆向量m：

```
m = [0.90, 0.10, 0.80, 0.20, 0.70, 0.00, 0.60, 0.10,
     0.80, 0.20, 0.90, 0.30, 0.60, 0.10, 0.70, 0.20,
     0.95, 0.00, 0.85, 0.10, 0.50, 0.30, 0.40, 0.20]
```

真实中，这里是24 行× 256 字节= 6144 字节。

5 投影成K 和V

Engram 借用Attention 的QKV 思想：Q 是当前隐藏状态h（含上下文），K 由m 投影得到（表示“记忆在说什么”），V 由m 投影得到（表示“如果采纳，注入什么”）。微缩示例定义两个超连接分支。

共享Value 投影（取每4 维平均）与分支Key：

```
V  = [(0.90+0.70+0.80+0.60+0.95+0.50)/6, (0.10+0.00+0.20+0.10+0.00+0.30)/6,
      (0.80+0.60+0.90+0.70+0.85+0.40)/6, (0.20+0.10+0.30+0.20+0.10+0.20)/6]
   = [0.742, 0.117, 0.708, 0.183]

K1 = [(0.90+0.70+0.80)/3, (0.10+0.00+0.20)/3, (0.80+0.60+0.90)/3, (0.20+0.10+0.30)
    /3]
```

```
   = [0.800, 0.100, 0.767, 0.200]

K2 = [(0.60+0.95+0.50)/3, (0.10+0.00+0.30)/3, (0.70+0.85+0.40)/3, (0.20+0.10+0.20)
    /3]
   = [0.683, 0.133, 0.650, 0.167]
```

6 上下文感知门控

当前位置的两个分支隐藏状态$h_1 = [0.5, 0.4, 0.6, 0.3]$，$h_2 = [0.4, 0.5, 0.5, 0.4]$。门控公式：$\alpha_i = \sigma\left(\frac{h_i \cdot K_i}{\sqrt{d}}\right)$，其中$d = 4, \sqrt{4} = 2$。

```
Branch 1: h1 * K1 = 0.5*0.8 + 0.4*0.1 + 0.6*0.767 + 0.3*0.2 = 0.9602
         0.9602 / 2 = 0.4801 -> alpha1 = sigmoid(0.4801) ~= 0.618
Branch 2: h2 * K2 = 0.4*0.683 + 0.5*0.133 + 0.5*0.65 + 0.4*0.167 = 0.7315
         0.7315 / 2 = 0.3658 -> alpha2 = sigmoid(0.3658) ~= 0.590
```

门控后的Value：$\tilde{V}_1 = \alpha_1 V = [0.459, 0.072, 0.438, 0.113]$，$\tilde{V}_2 = \alpha_2 V = [0.438, 0.069, 0.418, 0.108]$。门控值接近0.6，说明模型认为检索到的记忆与当前上下文基本一致。

7 后处理：Conv1D + SiLU

真实计算：`Y = SiLU(Conv1D(RMSNorm(V_gated))) + V_gated`。微缩示例假设RMSNorm 后数值不变，Conv1D 初始化为恒等映射。SiLU 公式：$\text{SiLU}(x) = x \cdot \sigma(x)$。

```
Branch 1: SiLU(0.459)~=0.281, SiLU(0.072)~=0.037, SiLU(0.438)~=0.266, SiLU(0.113)
    ~=0.060
         Y1 = [0.281+0.459, 0.037+0.072, 0.266+0.438, 0.060+0.113] = [0.740,
             0.109, 0.704, 0.173]
Branch 2: SiLU(0.438)~=0.266, SiLU(0.069)~=0.036, SiLU(0.418)~=0.252, SiLU(0.108)
    ~=0.057
         Y2 = [0.438+0.266, 0.069+0.036, 0.418+0.252, 0.108+0.057] = [0.704,
             0.105, 0.670, 0.165]
```

8 残差注入

最终把处理后的记忆残差加回隐藏状态：$h_1' = h_1 + Y_1$，$h_2' = h_2 + Y_2$。

```
h1' = [0.5+0.740, 0.4+0.109, 0.6+0.704, 0.3+0.173] = [1.240, 0.509, 1.304, 0.473]
h2' = [0.4+0.704, 0.5+0.105, 0.5+0.670, 0.4+0.165] = [1.104, 0.605, 1.170, 0.565]
```

9 最终预测“Paris”

把两个分支平均：`h_comb = (h1' + h2') / 2 = [1.172, 0.557, 1.237, 0.519]`。假设LM Head 中`w_paris = [1.0, 0.5, 1.1, 0.4]`，`w_capital = [0.2, 0.9, 0.3, 0.8]`。

```
logit_paris  = 1.172*1.0 + 0.557*0.5 + 1.237*1.1 + 0.519*0.4 = 3.020
logit_capital = 1.172*0.2 + 0.557*0.9 + 1.237*0.3 + 0.519*0.8 = 1.521

exp(3.020) ~= 20.49, exp(1.521) ~= 4.58
P(Paris) ~= 20.49 / (20.49 + 4.58) ~= 0.817
```

输出：`Paris`

10 真实规模

步骤	真实V4.1-Flash
压缩词表	99,092
哈希头	8
head_dim	256
每行字节	256 FP8
每token 每层取行数	24
每token 每层读取	6144 字节
Engram 层数	2
每层表行数	约3.84 亿
单层表大小	约94.6 GiB
两层总表	约189 GiB
推理吞吐损失	$< 3\%$

真实推理时，约189 GiB 的表不在HBM 中，而是放在主机内存或NVMe，通过后台RDMA 预取，与Transformer 计算重叠。每个token 位置在每层请求24 行，两层合计约12.4 KiB 内存流量。

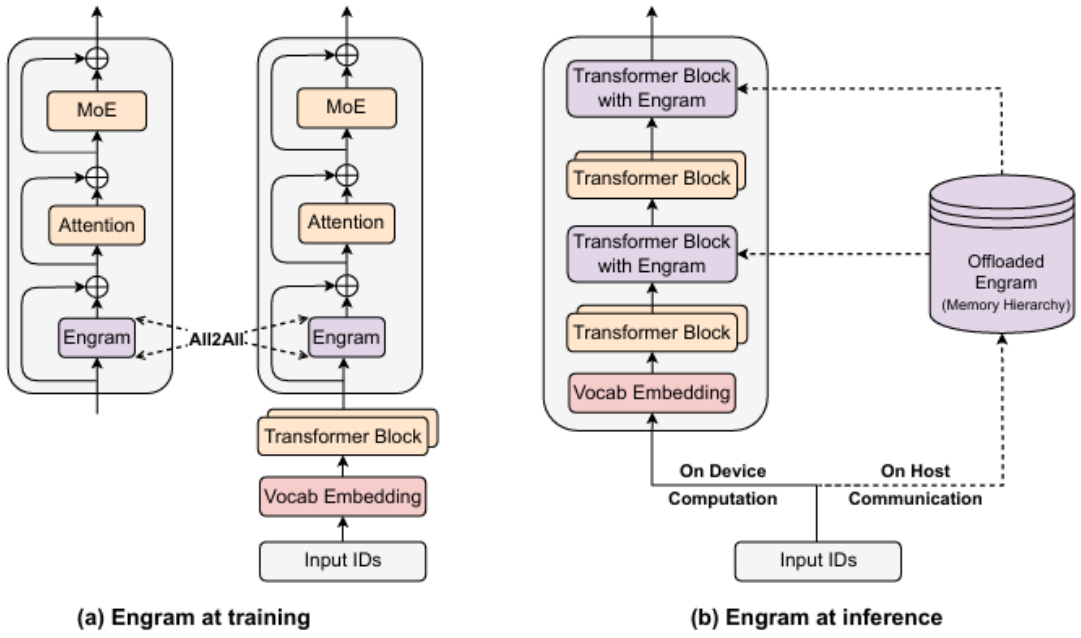


Figure 2: Engram 在训练与推理时的系统架构。（a）训练阶段：海量嵌入表分片存储，通过All2All 通信原语跨设备检索与分发梯度，实现容量的线性扩展；（b）推理阶段：Engram 表存放于主机内存或NVMe 中，通过Offloaded Engram Memory Hierarchy 管理，利用后台RDMA 预取与计算重叠，极大降低HBM 占用。